

UNIT CONVERTER USER INTERFACE

Hugh Williams

Problem Statement

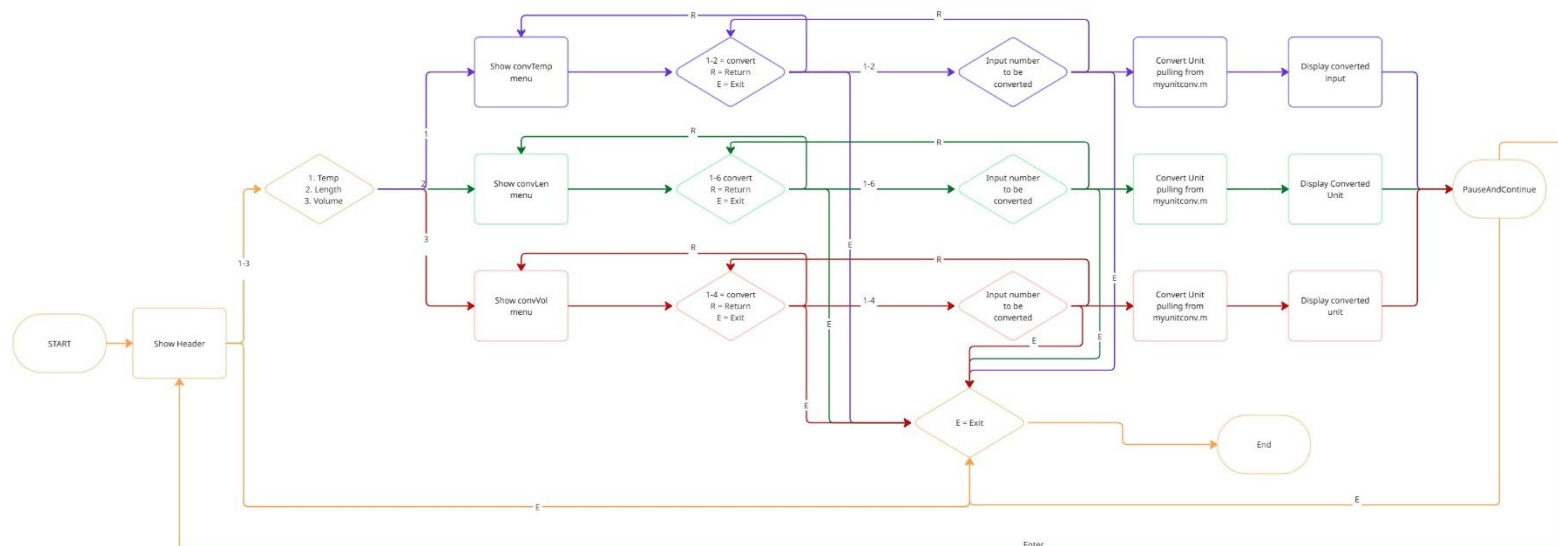
The problem that will be looked at with this program is the ease at which someone can convert units of measurement from metric to imperial and vice versa. It usually involves looking up the equations required to convert one value to its counterpart whether that being metres to feet or litres to imperial gallons. This can be prone to human error as well as time consuming when using what is found online. The aim of this program is to develop a text-based unit converter user interface to provide the user with a reliable, understandable and easy way to convert values between metric and imperial units. To achieve this goal the program will get the user to input a numerical value and a conversion type from a given menu, then receiving the instantaneous and accurate outputs from the program. To maximise a user-friendly program, it must include basic input validation and error handling when invalid inputs are entered by the user. This should optimise the users ease to operate the program so it is available for the use of users with varying capabilities when it comes to using the tool.

Algorithm Design



Flowchart.pdf

Open the pdf of the flow chart for better view

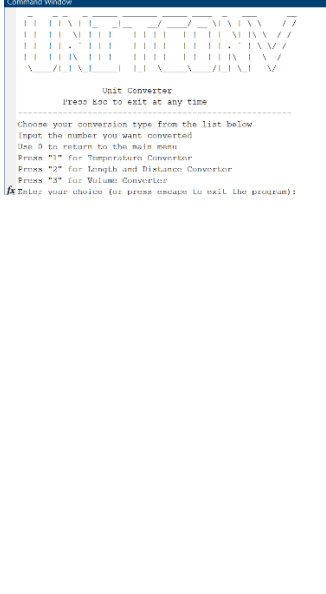
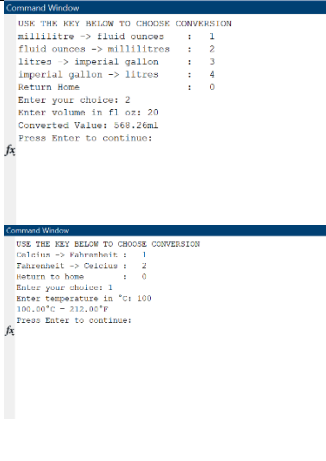
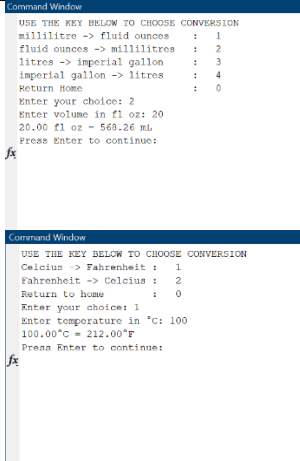
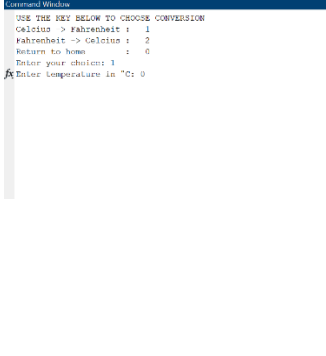
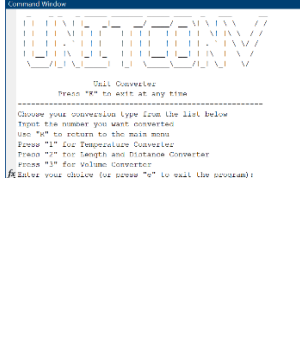


This flow chart shows how the program will operate. After starting the program, the user is offered 3 options of temperature, length and volume. After selecting which option, they want the user is then given a corresponding submenu where they can select with numbers which conversion they would like to do, to return to the previous page, or to exit the program. If choosing a conversion, the program will pull information from another program called myunitconv.m which contains the information necessary to do the conversion. Taking this information, the user interface turns the user's input into a converted unit. After which the user

can press enter to return to the main menu or E to exit the program. The programs input validation at each stage handles invalid selections and returns the user back to their current spot in the program. This flowchart also shows how all three-unit categories follow the same logic pattern so it is simple for the user to navigate the entire code. Entering E at any point also exits the program as shown in the flowchart.

Evidence of Testing

ERROR	CODE OUTPUT	FIX	NEW CODE OUTPUT
Error given by program when testing to see if it will take the function from conversion.m to convert Celsius to Fahrenheit. This is due to the code calling the whole file instead if individual functions within that file.	<pre> Command Window USE THE KEY BELOW TO CHOOSE CONVERSION Celsius -> Fahrenheit : 1 Fahrenheit -> Celsius : 2 Return to home : 0 Enter your choice: 1 Enter temperature in "C": 3 Discovered function or variable "celciusstofahrenheit". Error in userinterface:convert (line 103) result = celciusstofahrenheit(val); % calls function %~~~~~ Error in userinterface (line 41) convert %~~~~~ A>> </pre>	retain converter used in previous assessment, replacing conversion.m. call on the entire file and then using " call on the individual conversion.	<pre> Command Window USE THE KEY BELOW TO CHOOSE CONVERSION Celsius -> Fahrenheit : 1 Fahrenheit -> Celsius : 2 Return to home : 0 Enter your choice: 1 Enter temperature in "C": </pre>
While converting from cm to inches. When a input is put in and enter is pressed, it take user back to home screen.	<pre> Command Window _ _ _ _ _ / / / / / / / / / / / / / / / /___/___/___/___/___/___/___/___/___/___/___/___/___/___/___/ / / / / / / / / / / / / / / / /___/___/___/___/___/___/___/___/___/___/___/___/___/___/___/ / / / / / / / / / / / / / / / /___/___/___/___/___/___/___/___/___/___/___/___/___/___/___/ USE - SOURCECODE ===== Choose your conversion how long the line below Must be number you want conversion Press "0" for Temperature Conversion Press "1" for Length and Distance Conversion Press "2" for Mass Conversion Press your choice </pre>	Missing a while true above under the function so that the program continues after 1 conversion and doesn't immediately exit after that conversion, which was the error	<pre> Command Window USE THE KEY BELOW TO CHOOSE CONVERSION cm -> inch : 1 inch -> cm : 2 m -> feet : 3 feet -> m : 4 in -> miles : 5 miles -> km : 6 Return to home: 0 Enter your choice: 1 Enter length in cm: 20 Converted Value: 7.87 in Press enter to continue </pre>

Pressing escape on home screen doesn't close application like intended		Because Matlabs can't register the escape key I changed it so when 'E' is pressed the program closes. This is then added as a case in each section of the program so the user can close it at any time. Can't really show the output as it closes the program application so here is the toolbar after it is used->	
The output of the converted units doesn't have a standard sized format as the fprintf() varies slightly		change it so the fprintf() for all conversion function are put under its own case with the appropriate measurement value. This also makes it so they are the same to output answers in the same format style. Making it easier to understand.	
When 0 is entered as the input for the value converted it takes the user back to the home screen.		Changed the input to return to the main menu to 'r' or 'R' as they aren't used anywhere else and means that 0 can be entered as an individual input when calculation the conversion.	

Evaluation

This program fulfills all the requirements set in the task brief such as error handling, functionality and usability, and outputting the converted number in a neat and coherent manner in the command window. Using switch-case for use in navigating between conversion methods, the code allows for only certain inputs to work in the user-interface. If the input given isn't a case then it runs the 'otherwise' code and informs the user there is an error in their input and to select one of the valid inputs from the given menu.

The conversion methods broken up into three categories for similar unit conversions, making the user interface easy to understand and navigate. As this makes it easier for the user to find what they are looking for as well as making it easier to understand, it fulfills the requirements for usability.

Once the user has made their way to their method conversion, they are prompted to input the number they want converted into the corresponding unit type. After which they will receive an easily understandable and correct output for the converted output. E.g. $100^{\circ}\text{C} = 212^{\circ}\text{f}$. This meets the requirements of the program to output a converted number in a neat a coherent output.

The program calculates the conversion by calling on the file myunitconv.m which contains the information for converting between different units. This allows for the interface code to remain centred around the users' interactions, while the converting code is kept separate and reusable. Following the aim of the project to use good programming practices and code reuse.

Overall, the program fulfills the outlined requirements with its usability, functionality and structure, while also not breaching any constraints in what the program can't do or use.

